

APPSEC-07: Security Investigator and Davis CoPilot for Security

Series: APPSEC — Application Security | **Notebook:** 7 of 10 | **Created:** June 2026 |
Last Updated: 09/28/2026

Overview

When a security problem opens, an analyst's job is to follow the chain: which entities are affected, what else changed in the topology around that time, what's the most likely remediation? Dynatrace gives you two complementary surfaces for this work — the **Security Investigator** app (visual pivoting over entities and events) and **Davis CoPilot** (natural-language question-asking that generates DQL on demand).

This notebook covers when to reach for each, and the integration points back to the rest of the AppSec series.



Table of Contents

1. [1. Security Investigator Workflow](#)
 2. [2. Davis CoPilot for Security Questions](#)
 3. [3. DQL Generated On Demand](#)
 4. [4. Investigation Log as Audit Artifact](#)
 5. [5. Next Steps](#)
 6. [References](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	Gen3 SaaS with Grail; AppSec entitlement enabled
OneAgent	Full-Stack mode (or code-module attached) on monitored hosts
Read access	At minimum <code>environment:roles:view-security-problems</code> and <code>storage:security.events:read</code> — see APPSEC-09 for the full model
Background	APPSEC-01 (fundamentals + three-pillar framing)

1. Security Investigator Workflow

Investigator opens on a security problem and lets you pivot:

- **Affected entities** → the services, processes, hosts, containers touched by the vulnerability
- **Related security events** → other events on those entities in the same time window
- **Topology context** → Smartscape upstream / downstream so you can see blast radius
- **Davis problem correlation** → did a Davis problem (performance / availability) fire on the same entities at the same time?

The pivots are the value. A flat list of events is a triage queue; a pivoted view is an investigation.

Sources: [Investigations \(DT docs\)](#) — the investigation app for evidence-driven work on Grail data, including security: *"Investigations supports a wide range of evidence-driven investigations"* and *"Attach relevant findings as evidence, while preserving the investigation context."* (re-read 09/18/2026). **Softened:** the specific pivot set above is community-practice synthesis of the investigation surface — the Investigations page does not enumerate these pivots; verify them in your tenant.

2. Davis CoPilot for Security Questions

Davis CoPilot accepts natural-language questions and translates them into DQL or topology queries. Examples:

- "Which services were attacked the most in the last 24 hours?" → DQL over `security.events` filtered to `DETECTION_FINDING` from `Runtime Application Protection` (APPSEC-04 § 4)
- "Show me the security problems opened on payments services this week" → DQL + topology filter
- "Are any of these vulnerabilities reachable from the public internet?" → Investigator-style pivot

CoPilot is most useful when you can verbalize the question but the DQL is non-obvious — exactly the situation a security analyst hits often. Treat the generated DQL as a starting point; review and refine before pinning to a dashboard.

Softened: the example prompts and the CoPilot behavior described here are illustrative community practice, not documented on the Application Security pages — verify CoPilot's current generation behavior in your tenant. The AIOps series covers Davis CoPilot / Dynatrace Assist in depth.

3. DQL Generated On Demand

When CoPilot generates a query you want to keep, the path is: open it in a notebook (this surface), refine, then pin to a dashboard. The pattern below is the kind of query a well-formed prompt should produce — compare generated DQL against it, because generated

queries often skip the dedup step and count every periodic snapshot: *"Show me the open vulnerabilities grouped by severity and internet exposure."*

```
// Current open vulnerabilities by risk level and internet exposure (latest snapshot per vulnerability)
fetch security.events, from:-7d
| filter event.provider == "Dynatrace"
| filter event.category == "VULNERABILITY_MANAGEMENT"
| filter event.type == "VULNERABILITY_STATE_REPORT_EVENT"
| filter event.level == "VULNERABILITY"
| dedup {vulnerability.display_id}, sort:{timestamp desc}
| filter vulnerability.resolution.status == "OPEN"
| filter vulnerability.mute.status == "NOT_MUTED"
| summarize open = count(), by:{vulnerability.risk.level, vulnerability.davis_assessment.exposure_status}
| sort open desc
```

Validation note: validated for syntax and field names on 09/18/2026 and executes cleanly; the validation tenant has no RVA data, so it returned 0 rows there.

Sources: [Vulnerability events \(DT semantic dictionary\)](#) —

vulnerability.davis_assessment.exposure_status examples NOT_AVAILABLE ; NOT_DETECTED ; PUBLIC_NETWORK ; ADJACENT_NETWORK , and *"a deduplicated latest snapshot per vulnerability reflects present exposure"* (re-read 09/18/2026). **Dictionary:** vulnerability.davis_assessment.exposure_status (stable), vulnerability.risk.level (stable), vulnerability.resolution.status (stable), vulnerability.mute.status (stable), read 09/18/2026; no row for vulnerability.public_exposure under filter startsWith(name, "vulnerability.public") , read 09/18/2026 (control: filter startsWith(name, "vulnerability")) → 70 rows).

4. Investigation Log as Audit Artifact

In community practice in regulated environments, the investigation itself is treated as an audit artifact, with two habits:

- 1. **Open each investigation in a notebook**, not just the Investigator UI. The notebook captures the queries, the reasoning, and the conclusion as a durable record.
- 2. **Attach the notebook to the ticketing system** (Jira/ServiceNow link) so the security-problem record links to the investigation record. APPSEC-08 covers the workflow patterns.

This is more discipline than tooling. Both surfaces support it; the question is whether the team commits to using them this way.

Sources: [Investigations \(DT docs\)](#) — *"Attach relevant findings as evidence, while preserving the investigation context."* and *"view your investigation history"* (re-read 09/28/2026).

5. Next Steps

- 1. Open a recent security problem in Investigator. Walk the pivots end-to-end.
- 2. Ask CoPilot for one of the example prompts above. Review the generated DQL.
- 3. Read **APPSEC-08** to wire the investigation outcome to a remediation ticket.
- 4. Read **APPSEC-09** — CoPilot can only generate queries against data the user has IAM access to; the policy model bounds the assistant's reach.

References

Source	Coverage
Investigations (DT docs)	The investigation app: queries, evidence, history, collaboration
Vulnerability events (DT semantic dictionary)	Exposure and risk fields used in § 3
Application Security (DT docs)	AppSec hub

 **DISCLAIMER:** This information was AI generated and is provided "as-is" without warranty. It was produced as an independent, community-driven project and **not supported by Dynatrace**. Always refer to official [Dynatrace documentation](#) for the most current information.